CHAPTER 08

Software Configuration Management

ACRONYMS

CR	Change Request
CCB	Configuration Control Board
CI	Configuration Item
CM	Configuration Management
CMDB	Configuration Management Database
CMMI	Software Engineering Institute's Capability Maturity Model Integration
FCA	Functional Configuration Audit
MBX	Model Based Experience
PCA	Physical Configuration Audit
QA	Quality Assurance
SBOM	Software Bill Of Materials
SCCB	Software Configuration Control Board
SCR	Software Change Request
SCI	Software Configuration Item
SCM	Software Configuration Management
SCMP	Software Configuration Management Plan
SCSA	Software Configuration Status Accounting
SLCP	Software Life Cycle Process
SQA	Software Quality Assurance
V&V	Verification And Validation
VDD	Version Description Document

INTRODUCTION

Software configuration management (SCM) is formally defined as "the process of applying configuration management [CM] throughout the software life cycle to ensure the completeness and correctness of CIs [configuration items]," with CM defined as "a discipline applying technical and administrative direction and surveillance to identify and document the functional and physical characteristics of a configuration item, control changes to those characteristics, record and report change processing and implementation status, and verify compliance with specified requirements" [1]. SCM is a software life cycle process (SLCP) that supports project management, development and maintenance activities, quality assurance (QA) activities, and the customers and users of the end product.

The concepts of CM apply to all items controlled, although some differences exist between implementing hardware CM and implementing software CM. CM applies equally to iterative and incremental software development methodology.

SCM is closely related to software quality assurance (SQA). As defined in the Software Quality KA, SQA processes ensure that the software products and processes in the project life cycle conform to their specified requirements by requiring software engineers to plan, enact and perform a set of activities that demonstrate that those specifications are built into the software. SCM activities support these SQA goals through software configuration activities (presented later in this chapter). The configuration audit activity can be described as a review of CIs and is closely related to the reviews defined in the quality plan.

The SCM activities should operationalize SCM process management and planning, software configuration identification, software configuration change control, software configuration status accounting (SCSA),

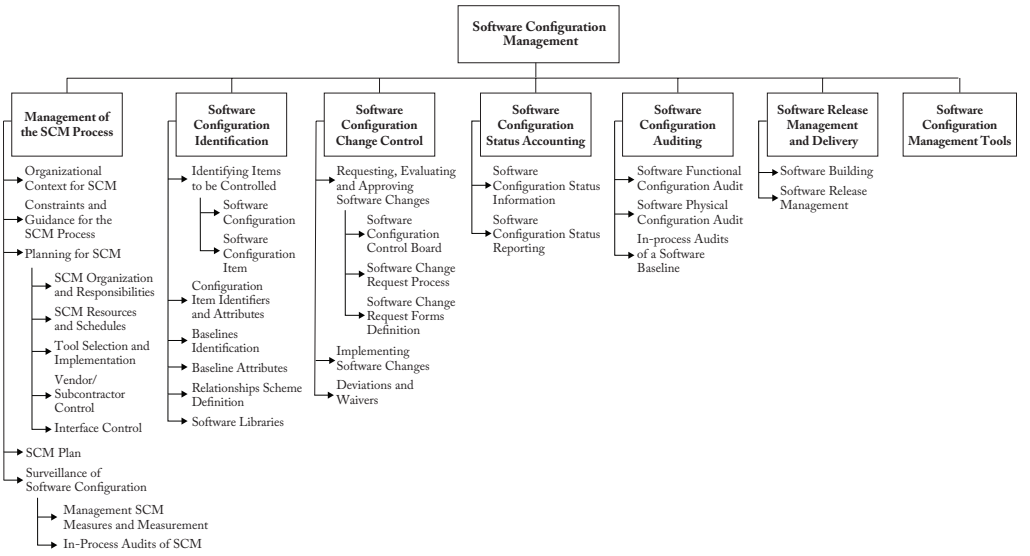


Figure 8.1. Breakdown of Topics for the Software Configuration Management KA.

software configuration auditing, and software release management and delivery. This operationalization:

1. Determines what is expected to be under control during project development
2. Identifies and records who developed what CI as well as when and where it is allocated
3. Allows controlled changes
4. Tracks CIs' relationships to show how changes that affect one CI might affect other CIs
5. Keeps CI versions under control
6. Ensures that the quality of the CIs delivered meets the requirements for intended use

The SCM KA is related to all other KAs because SCM's object is the artifact produced and used throughout the software engineering process.

BREAKDOWN OF TOPICS FOR SOFTWARE CONFIGURATION MANAGEMENT

Figure 8.1 shows the breakdown of topics for the SCM KA.

1. Management of the SCM Process [2, c6, c7]

SCM controls the evolution and integrity of a product by identifying its elements (known as CIs); managing and controlling change; and verifying, recording and reporting on configuration information. From the software engineer's perspective, SCM facilitates development and change implementation activities. Successful SCM implementation requires careful planning and management, which requires a strong understanding of the organizational context for, and the constraints placed on, the design and implementation of the SCM process. The SCM plan can be developed once for the organization and then adjusted as needed for individual projects.

1.1 Organizational Context for SCM
[2, c6, ann. D] [3*, Introduction] [4*, c25]

To plan an SCM process for a project, it is necessary to understand the organizational context and the relationships among organizational elements. SCM interacts not just

with the particular project but also with several other areas of the organization.

The organizational elements responsible for software engineering supporting processes might be structured in various ways. The overall responsibility for SCM often rests with a distinct part of the organization or with a designated individual. However, responsibility for certain SCM tasks might be assigned to other parts of the organization (such as the development division).

Software is frequently developed as part of a larger system containing hardware and firmware elements. In this case, SCM activities take place in parallel with hardware and firmware CM activities and must be consistent with system-level CM. Note that firmware contains hardware and software; therefore, both hardware and software CM concepts apply.

SCM might interface with an organization's QA activity on issues such as records management and nonconforming items. Regarding the former, project records subject to provisions of the organization's QA program might also be under SCM control. The QA team is usually responsible for managing nonconforming items. However, SCM might assist with tracking and reporting on software configuration items (SCIs) in this category.

Perhaps the closest relationship is with the software development and maintenance organizations. It is within this context that many of the software configuration control tasks are conducted. Frequently, the same tools support development, maintenance, and SCM purposes.

1.2 Constraints and Guidance for the SCM Process

[2, c6, ann. D, ann. E] [3*, c2, c5] [5, c19s2.2]

Constraints affecting, and guidance for, the SCM process come from many sources. Policies and procedures set forth at corporate or other organizational levels might influence or prescribe the design and implementation of the SCM process for a project. In addition, the contract between the acquirer and the supplier might contain provisions affecting the SCM process (e.g., certain configuration audits might be required, or the contract might specify that certain items be placed under CM). When the software to be developed could affect public safety, external regulatory bodies may impose constraints. Finally, the SLCP chosen for a software project and the level of formalism selected for implementation will also affect SLCP design and implementation.

Software engineers can also find guidance for designing and implementing an SCM process in "best practice," as reflected in the software engineering standards issued by the various standards organizations. (See Appendix B for more information about these standards.)

1.3 Planning for SCM

[2, c6, ann. D, ann. E] [3*, c23] [4*, c25]

SCM process planning for a project should be consistent with the organizational context, applicable constraints, commonly accepted guidance and the nature of the project (e.g., size, safety criticality and security). The major activities covered in the plan are software configuration identification, software configuration control, SCSA, software configuration auditing, and software release management and delivery. In addition, issues such as organization and responsibilities, resources and schedules, tool selection and implementation, vendor and subcontractor control, and interface control are typically considered. The planning activity's results are recorded in an SCM plan (SCMP), which is subject to SQA review and audit.

Branching and merging strategies should be carefully planned and communicated because they affect many SCM activities. SCM defines a branch as a set of evolving source file versions [1]. Merging consists of combining different changes to the same file [1]. This typically occurs when more than one person changes a CI. There are many

branching and merging strategies in common use. (See the Further Readings section for additional discussion.)

The software development life cycle model chosen (see Software Life Cycle Models in the Software Engineering Process KA) also affects SCM activities, and SCM planning should consider this. For instance, many software development approaches use continuous integration, which is characterized by frequent build-test-deploy cycles. Clearly, SCM activities must be planned accordingly.

1.3.1 SCM Organization and Responsibilities

[2, ann. Ds5-6] [3*, c10-11] [4*, c25]

Organizational roles must be clearly identified to prevent confusion about who will perform specific SCM activities or tasks. These responsibilities must also be assigned to organizational entities; this can be made clear by the responsible individual's title or by designating the organizational division or section in addition to the individual responsible within that section. The overall authority and reporting channels for SCM should also be identified, although this might be accomplished at the project management or the QA planning stage.

1.3.2 SCM Resources and Schedules

[2, ann. Ds8] [3*, c23]

Planning for SCM identifies the resources — including staff and tools — involved in carrying out SCM activities and tasks. It also identifies the necessary sequences of SCM tasks and establishes each task's place in the project schedule and its position relative to milestones established at the project management planning stage. Any training requirements for implementing the plans and training new staff members are also specified.

1.3.3 Tool Selection and Implementation

[3*, c26s2, c26s6]

As in any area of software engineering, the selection and implementation of SCM tools should be carefully planned. The following questions should be considered:

- Organization: What motivates tool acquisition from an organizational perspective?
- Tools: Can we use commercial tools, or do we need to develop our own tools specifically for this project?
- Environment: What constraints are imposed by the organization and its technical context?
- Legacy: How will projects use (or not use) the new tools?
- Financing: Who will pay for the tools' acquisition, maintenance, training and customization?
- Scope: How will the new tools be deployed — for instance, through the entire organization or only on specific projects?
- Ownership: Who is responsible for introducing new tools?
- Future: What is the plan for the tools' use in the future?
- Change: How adaptable are the tools?
- Branching and merging: Are the tools' capabilities compatible with planned branching and merging strategies?
- Integration: Do the various SCM tools integrate among themselves? Do they integrate with other tools in use in the organization?
- Migration: Can the repository maintained by the version control tool be ported to another version control tool while maintaining the complete history of the CIs it contains?

SCM requires a set of tools instead of a single tool. Such tool sets are sometimes called *workbenches*. As part of the tool selection planning effort, the team must determine whether the SCM workbench will be *open* (tools from different suppliers will be used in different SCM process activities) or *integrated* (elements of the workbench are designed to work together).

Organization size and the type of projects involved may also affect tool selection. (See SCM Tools, section 7 of this document)

1.3.4 Vendor/Subcontractor Control

[2, c13] [3*, c13s9-c14s2]

A software project might acquire or use purchased software products, such as compilers or other tools. SCM planning considers whether and how these items will be managed with configuration control (e.g., integrated into the project libraries) and how changes or updates will be evaluated and managed.

Similar considerations apply to subcontracted software. When a project uses subcontracted software, both the SCM requirements to be imposed on the subcontractor's SCM process and the means for monitoring compliance need to be established. The latter includes determining what SCM information must be available for effective compliance monitoring.

1.3.5 Interface Control

[2, c12] [3*, c23s4]

When a software item interfaces with another software or with a hardware item, a change to either item can affect the other. Planning for the SCM process considers how the interfacing items will be identified and how changes to the items will be managed and communicated. The SCM role may be part of a larger, system-level process for interface specification and control involving interface specifications, interface control plans and interface control documents. In this case, SCM planning for interface control takes place within the context of the system-level process.

1.4 SCM Plan

[2, ann. D] [3*, c23]

The results of SCM planning for a given project are recorded in an SCMP, a "living document" that serves as a reference for the SCM process. The SCMP is maintained (updated and approved) as necessary during the software life cycle. For teams to implement an SCMP, they'll typically need to develop a number of more detailed, subordinate procedures to define how specific requirements will be met during day-to-day activities (e.g., which branching strategies will be used, how frequently builds will occur, how often automated tests of all kinds will be run).

Guidance on creating and maintaining an SCMP, based on the information produced by the planning activity, is available from a number of sources, such as [2]. This reference provides requirements for information to be contained in an SCMP. An SCMP should include the following sections:

- Introduction (purpose, scope, terms used)
- SCM Management (organization, responsibilities, authorities, applicable policies, directives, procedures)
- SCM Activities (configuration identification, configuration control, etc.)
- SCM Schedules (coordination with other project activities)
- SCM Resources (tools, physical resources and human resources)
- SCMP Maintenance

1.5 Monitoring of Software Configuration Management

[3*, c11s3]

After the SCM process has been implemented, some surveillance may be necessary to ensure that the SCMP provisions are properly carried out. The plan is likely to include specific SQA requirements to ensure compliance with specified SCM processes and procedures. The person responsible for SCM ensures that those with the assigned responsibility perform the defined SCM tasks correctly. As part of a compliance auditing activity, the SQA authority might also perform this surveillance.

Using integrated SCM tools with process control capability can make the surveillance task easier. Some tools facilitate process compliance while providing flexibility for the software engineer to adapt procedures. Other tools enforce a specific process, leaving the software engineer with less flexibility. Surveillance requirements and the level of flexibility provided to the software engineer are important considerations in tool selection.

1.5.1 SCM Measures and Measurement

[3*, c9s2, c25s2-s3]

SCM measures can be designed to provide specific information on the evolving product, but they can also provide insight into how well the SCM process functions and identify opportunities for process improvement. SCM process measurements enable teams to monitor the effectiveness of SCM activities on an ongoing basis. These measurements are useful in characterizing the current state of the process and providing a basis for comparison over time. Measurement analysis may produce insights that lead to process changes and corresponding updates to the SCMP.

Software libraries and the various SCM tool capabilities enable teams to extract useful information about SCM process characteristics (as well as project and management information). For example, information about the time required to accomplish various types of changes would be useful in evaluating criteria for determining what levels of authority are optimal for authorizing certain changes and in estimating the resources needed to make future changes.

Care must be taken to keep the surveillance focused on the insights that can be gained from the measurements, not on the measurements themselves. Software process and product measurement is further discussed in the Software Engineering Process KA. Software measurement programs are described in the Software Engineering Management KA.

1.5.2 In-Process Audits of SCM

[3*, c1s1]

Audits can be carried out during the software engineering process to investigate the status of specific configuration elements or to assess the SCM process implementation. In-process SCM auditing provides a more formal mechanism for monitoring selected aspects of the process and may be coordinated with the SQA function. (See Software Configuration Auditing.)

2. Software Configuration Identification

[2, c8]

Software configuration identification identifies items to be controlled, establishes identification schemes for the items and their versions, and establishes the tools and techniques to be used in acquiring and managing controlled items. These activities provide the basis for other SCM activities.

2.1 Identifying Items to Be Controlled

[2, c8s2.2]

A first step in controlling change is identifying the software items to be controlled. This involves understanding the software configuration within the context of the system configuration, selecting SCIs and developing a strategy for labeling software items.

2.1.1. Software Configuration

Software configuration is the functional and physical characteristics of hardware or software as set forth in technical documentation or achieved in a product. It can be viewed as part of an overall system configuration.

2.1.2 Software Configuration Item

[2, c8s2.1] [3*, c9]

A CI is an item or aggregation of hardware, software or both, designed to be managed as a single entity. An SCI is a software entity that has been established as a CI [1]. The SCM controls various items in addition to the code itself. Software items with potential to become SCIs include plans, specifications and design documentation, testing materials, software tools, source and executable code, code libraries, data and data dictionaries, and documentation for installation, maintenance, operations and software use.

Selecting SCIs is an important process in which a balance must be achieved between providing adequate visibility for project control purposes and providing a manageable number of controlled items.

2.2 Configuration Item Identifiers and Attributes

[2, c8s2.3, c8s2.4] [3*, c9]

Status accounting activity (explained later) gathers information about CIs while they are developed. The CIs' scheme is defined in order to establish what information must be gathered and tracked for each CI. Unique identifiers and versions are tracked.

An example scheme may include the following:

CI name
CI unique identifier
CI description
CI date(s)
CI type
CI owner

The CI's unique Identifier can use significant or nonsignificant codification. An example of significant codification could be XX-YY, where XX is the iteration abbreviation (in case of using an iterative development method) and YY is the CI abbreviation.

2.3 Baseline Identification

[2, c8s2.5.4, c8s2.5.5, c8s2.5.6]

A software baseline is a formally approved version of a CI (regardless of media type) that is formally designated and fixed at a specific time during the CI's life cycle. The term also refers to a particular version of an agreed-upon SCI. The baseline can be changed only through formal change control procedures. A baseline, with all approved changes to the baseline, represents the current approved configuration. A baseline consists of one or more related CIs.

2.4 Baseline Attributes

[2, c8s2.5.4]

Baseline attributes are used in the status accounting activity and specify information about the baseline established.

Example baseline attributes may include the following:

Baseline name
Baseline unique identifier
Baseline description
Baseline date of creation
Baseline CIs

2.5 Relationships Scheme Definition

[3*, c7s4]

Relationships provide the connections required to create and sustain structure. The ability to communicate intent and manage the results are significantly enhanced when effective relationships (structuring) are in place (e.g., model-based experience (MBX) platforms). Relationship information exchange and interoperability are needed to support the applicable relationship types. The status accounting activity is responsible for gathering information about relationships among CIs.

Common types of relationships can be described according to the following schemes:

Dependencies: CI-1 and CI-2 depend mutually on each other.

Example: CI-1 depends on C1-2, and vice versa, for instance a class model depends on a sequence diagram, because any change on any of both types of models, affect the other.

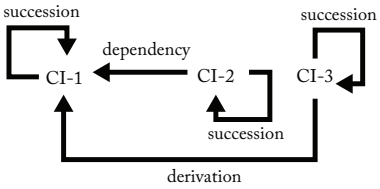
CI-1 Code	CI-2 Code	Date

Derivation: One CI derives from another, typically in a sequential relationship, not because of a lack of resources to handle both CIs but because of a constraint that requires that, for instance, CI-1 is completed before CI-2 is developed.

Example: CI-1 derives from CI-2.

CI-1 Code	CI-2 Code	Date

Succession: Software items evolve as a software project proceeds. A software item version is an identified instance of an item. It can be thought of as a state of an evolving item. This is what the succession relationship



Successions Records: According to the scheme defined for succession relationships, the next table gives the date when each CI was created (three first rows), and the fourth row indicates a change made on the CI-1 on 10/05/2021, where the current version was 1 and created new version is 2.

CI-1	–	1	10/01/2021
CI-2	–	1	10/04/2021
CI-3	–	1	10/03/2021
CI-1	1	2	10/05/2021

Dependency Record: According to the scheme defined for dependencies CI-1 and CI-2 have a dependency relationship created the day CI-2 was developed.

CI-1	CI-2	10/04/2021

Derivation Record: According to the scheme defined for derivation, CI-3 derives from CI-1 and this relationship came up the day CI-3 was created.

CI-3	CI-1	10/03/2021

Figure 8.2. Example of reported relationships

reflects, and it is reflexive in that each CI has this relationship with itself. The first succession comes up the first time a CI is created. Each time it is changed, a new succession comes up, and tracking these successions is the way to track CI versions.

Example: CI versions along a timeline.

CI Code	Current Version	Next Version	Date

Variants are program versions resulting from engineered alternative options. This type of relationship is not as common as the type of relationships described above because it is more expensive to maintain.

The decision on what relationships to track throughout the project is important because tracking some relationships can require extra work. On the other hand, tracking such relationships can facilitate decisions on change requests (CRs) for a CI.

Relationships between CIs can be tracked in a Software Bill of Materials (SBOM). An SBOM is a formal record containing the details and supply chain relationships of the CIs used in building software. CIs in an SBOM are frequently referred to as *components*. Components can be source code, libraries, modules and other artifacts; they can be open source or proprietary, free or paid; and the data can be widely available or access-restricted.

A simple example of the relationships among three CIs in an SBOM, called CI-1, CI-2 and CI-3, is illustrated in Figure 8.2.

2.6 Software Libraries

[2, c8s2.5] [3*, c1s3]

A software library is a controlled collection of source code, scripts, object code, documentation and related artifacts. Requirements and test cases are stored in a repository and should be linked with the code baselines developed. Source code is stored in a version control system, which provides traceability and security for the baselines developed. Multiple development streams are supported in version control systems linked to the binary objects (e.g., object code) derived during the build process. These binary objects are typically stored in a repository that should contain cryptographic hashes used to perform the physical configuration audit (PCA).

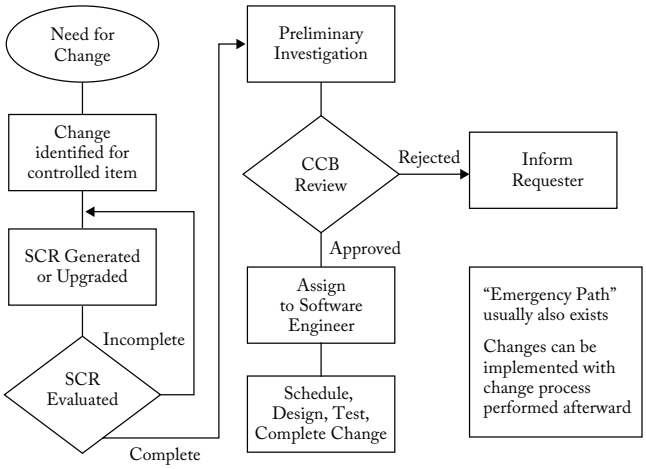


Figure 8.3. Flow of a Change Control Process

The definitive media library contains the release baseline(s) of the artifacts that can be deployed to the test, stage and production systems.

The release management process depends on these software libraries to manage the artifacts deployed. In terms of access control and the backup facilities, security is a key aspect of library management.

3. Software Configuration Change Control [2, c9] [3*,c8] [4*, c25s3] [5, c11.s3.3]

Software configuration change control is concerned with changes required to CIs during the software life cycle. It covers the process for determining what changes to make, the authority for approving certain changes, support for implementing those changes, and the concept of formal deviations from project requirements as well as waivers of them. Information derived from these activities is useful in measuring change traffic and breakage, as well as aspects of rework.

Given that change to CIs can follow specific rules depending on the industrial sector, area, company, etc., it is very important to identify those rules in the context of the software project for which the SCM process is being developed and to adhere strictly to those rules. The rest of this section can be useful when no specific rules regarding change control exist in the company or the industrial sector where the software project under development is allocated.

3.1 Requesting, Evaluating, and Approving Software Changes [2, c9s2.4] [3*, c11s1] [4*, c25s3]

The first step in managing changes to controlled items is determining what changes to make. The software change request (SCR) process (Figure 8.3) provides formal procedures for submitting and recording CRs; evaluating the potential cost and impact of a proposed change; and accepting, modifying, deferring or rejecting the proposed change. A CR is a request to expand or reduce the project scope; modify policies, processes, plans or procedures; modify costs or budgets; modify implemented code; or revise schedules [1]. Requests for changes to SCIs may be originated by anyone at any point in the software life cycle and may include a suggested solution and requested priority. One source of a CR is the initiation of corrective action in response to problem reports. Regardless of the source, the type of change (e.g., defect or enhancement) is usually recorded on the SCR.

Recording of the SCR enables the software engineers to track defects and collect change activity measurements by change type. Once an SCR is received, a technical evaluation (also known as an *impact analysis*) is performed to determine the extent of the modifications necessary should the CR be accepted. A good understanding of the relationships among software (and, possibly, hardware) items is important for this task. The information recorded about the CIs' relationships could be useful for making decisions affecting any CI, given the potential impact on other CIs. Finally, an established authority — commensurate with the affected baseline, the SCI involved and the nature of the change — will evaluate the CR's technical and managerial aspects and accept, modify, reject or defer the proposed change.

3.1.1 Software Configuration Control Board

[2, c9s2.2] [3*, c11s1] [4*, c25s3]

The authority for accepting or rejecting proposed changes rests with an entity known as a configuration control board (CCB). In smaller projects, this authority may reside with the leader or an assigned individual rather than a multi-person board. There can be multiple levels of change authority depending on a variety of criteria — such as the criticality of the item involved, the nature of the change (e.g., impact on budget and schedule), or where the project is in the life cycle. The composition of the CCBs used for a system varies depending on these criteria (but an SCM representative is always present). All stakeholders appropriate to the CCB level are represented. When a CCB's scope of authority is limited to software, the board is known as a Software Configuration Control Board (SCCB). The CCB's activities are subject to software quality audits or reviews.

3.1.2 Software Change Request Process

[3*, c1s4, c8s4] [4*, c25s3]

An effective SCR process requires the use of supporting tools and procedures for originating CRs, enforcing the change process flow, capturing CCB decisions and reporting change process information. Linking this tool capability with the problem-reporting system can facilitate the problem resolution tracking and how quickly solutions are developed.

3.1.3 Software Change Request Forms Definition

[2, c9s2.3, c9s2.5] [3*, c8s4] [4*, c25s3]

A CR application must include the following:

- A CR form, which must describe the request and give the rationale for it
- A change certification form (necessary if the CR is granted)

These forms can be managed through the corresponding supporting tool, but humans are responsible for designing the forms.

3.2 Implementing Software Changes

[4*, c25s3]

Approved SCRs are implemented using the defined software procedures per the applicable schedule requirements. Because a number of approved SCRs might be implemented simultaneously, a means for tracking which SCRs are incorporated into particular software versions and baselines must be provided. At the end of the change process, completed changes may undergo configuration audits and software quality verification, which includes ensuring that only approved changes have been made. The SCR process typically documents the change's SCM and other approval information.

Changes may be supported by source code version control tools. These tools allow a team of software engineers, or a single software engineer, to track and document changes to the source code. These tools provide a single repository for storing the source code, so they can prevent more than one software engineer from editing the same module at the same time, and they record all changes made to the

source code. Software engineers check modules out of the repository, make changes, document the changes, and then save the edited modules in the repository. If needed, changes can also be discarded, restoring a previous baseline. More powerful tools can support parallel development and geographically distributed environments. These tools may manifest as separate, specialized applications under an independent SCM group's control. They may also appear as an integrated part of the software engineering environment. Finally, they may be as elementary as a rudimentary change control system that is provided with an operating system.

3.3 Deviations and Waivers

[1, c3]

The constraints imposed on a software engineering effort or the specifications produced during the development activities might contain provisions that those working on the project find cannot be satisfied at the designated point in the life cycle. A *deviation* is a written authorization granted before the manufacture of an item to depart from a particular performance or design requirement for a specific number of units or a specific period of time. A *waiver* is a written authorization to allow a CI or other designated item in response to an issue found during production or after the project is submitted for inspection to depart from specified requirements when the CI or project is nevertheless considered suitable for use, either as it is or after rework via an approved method. In these cases, a formal process is used to gain approval for deviations from or waivers of the provisions.

4. Software Configuration Status Accounting

[2, c10] [3*, c9] [5, c11s3.4]

SCSA is an activity of CM consisting of recording and reporting information needed to manage a configuration effectively regarding CIs, baselines and relationships among CIs. This activity must be done by following the logical schemes defined in the activity configuration identification for CIs, baselines and relationships for gathering information.

4.1 Software Configuration Status Information

[2, c10s2.1]

The SCSA activity designs and operates a system for capturing, verifying, validating and reporting necessary information as the life cycle proceeds. As in any information system, the configuration status information to be managed for the evolving configurations must be identified, collected and maintained. In addition, the information itself should be secured where relevant. SCSA information and measurements are needed to support the SCM process and to meet the configuration status reporting needs of management, software engineering, security, performance and other related activities.

The types of information available include but are not limited to the following:

- Ongoing and approved configuration identification
- Current implementation status of changes
- Impacted CIs and related systems
- Deviations and waivers
- Verification and validation (V&V) activities

Automated tools support SCSA as tasks are performed, and reporting is available in a user-friendly format.

4.2 Software Configuration Status Reporting

[2, c10s2.4] [3*, c1s5, c9s1]

Reported information can be used by various organizational and project elements — including the development team, operations, security, the maintenance team, project management, software quality activities teams and others. Reporting can take many forms: automated reports, ad hoc queries to answer specific questions, and regular production of predesigned reports, including those developed to meet security, legal or regulatory

requirements. In other words, information produced by the SCSA activity throughout the life cycle can be used to satisfy QA and security and to provide evidence of compliance with regulations, governance requirements, etc.

In addition to reporting the configuration's current status, the information obtained by the SCSA can serve as a basis for various measurements.

Modern SCM includes a wider scope of information, including but not limited to the following:

- Indicators of integrity (e.g., MAC (Message Authentication Code) SHA1 (Secure Hash Algorithm), MD5 (Message Digest))
- Indicators of security status (e.g., governance risk and compliance)
- Evidence of V&V activities (e.g., requirements completion)
- Baseline status
- The number of CRs per SCI
- The average time needed to implement a CR

5. Software Configuration Auditing [2, c11] [5, c11s3.5]

A software audit is an independent examination of a work product or set of work products to assess technical, security, legal and regulatory compliance with specifications, standards, contractual agreements or other criteria [1]. Audits are conducted according to a well-defined process comprising various auditor roles and responsibilities. Because of this complexity, each audit must be carefully planned. An audit can require a number of individuals to perform various tasks over a fairly short time. Tools to support the planning and conduct of an audit can greatly facilitate the process.

Software configuration auditing determines the extent to which an item satisfies requirements for functional and physical characteristics. Informal audits can be conducted at key points in the life cycle. Two types of formal audits might be required by the governing contract (e.g., a contract covering critical software): the functional configuration audit (FCA) and the PCA. Successful completion of these audits can be a prerequisite for establishing the product baseline.

5.1 Software Functional Configuration Audit [2, c11s2.1]

The software FCA ensures that the audited software item is consistent with its governing specifications. The software V&V activities' output (see Verification and Validation in the Software Quality KA) is a key input to this audit.

5.2 Software Physical Configuration Audit [2, c11s2.2]

The software PCA ensures that the design and reference documentation are consistent with the as-built software product.

5.3 In-Process Audits of a Software Baseline [2, c11s2.3]

Audits can be carried out during the development process to investigate the status of specific configuration elements. In-process audits can be applied to all baseline items to ensure that performance is consistent with specifications or that evolving documentation continues to be consistent with the developing baseline item.

This task applies to every single CI to be approved as part of a baseline. The audit consists of reviewing the CI to determine whether it satisfies requirements. How to conduct the review and the expected result must be described in the quality plan or if there is no quality plan, defined for the software configuration auditing activity.

Continuous reviews of CIs identified in the configuration identification activities help verify conformance to governance and regulatory requirements.

Configuration auditing reviews take place throughout project development, whenever a CI must be reviewed.

6. Software Release Management and Delivery

[2, c14] [3*, c8s2] [4*, c25s4]

In this context, *release* refers to distributing software and related artifacts outside the development activity, including internal releases and distribution to customers. When different versions of a software item are available for delivery (such as versions for different platforms or versions with varying capabilities), re-creating specific versions and packaging the correct materials for version delivery are frequently necessary. The software library is a key element in accomplishing release and delivery tasks.

6.1 Software Building [4*, c25s2]

Software building constructs the correct versions of SCIs, using the appropriate configuration data, into a software package for delivery to a customer or other recipient such as a team performing testing. For systems with hardware or firmware, the executable program is delivered to the system-building activity. Build instructions help ensure that the proper build steps are taken in the correct sequence. In addition to building software for new releases, SCM must usually be able to reproduce previous releases for recovery, testing, maintenance or additional release purposes.

Software is built using supporting tools, such as compilers. For example, if it is necessary to rebuild an exact copy of a previously built SCI, supporting tools and associated build instructions must be under SCM control to ensure the availability of the correct versions of the tools.

Tool capability is useful for selecting the correct versions of software items for a target environment and automating the process of building the software from the selected version and configuration data. This tool capability is necessary for projects with parallel or distributed development environments. Most software engineering environments provide this capability. However, these tools vary in complexity; some require the software engineer to learn a specialized scripting language, while others use a more graphics-oriented approach that hides much of the complexity of an "intelligent" build facility.

The build process and products are often subject to software quality verification. Outputs of the build process might be needed for future reference. They may become records of quality, security, or compliance with organizational or regulatory requirements. The SBOM listing the artifacts included in the build is an important CM output.

In continuous integration, software building is performed automatically when changes to CIs are committed to a source control repository. Tools running on a local or cloud-based server monitor the project's source control system and initiate a pipeline of steps to be undertaken every time a change is committed to a particular branch or area of the source code repository. The tool is configured to retrieve a fresh copy of the complete source code for the project and execute the necessary commands to compile and link the code. This configuration is often combined with steps to validate coding standards via automated static analysis, execute unit tests and determine code coverage metrics, or extract documentation from the source code. The resulting artifacts are then deployed through the Release Management process.

6.2 Software Release Management [4*, c25s2]

Software release management encompasses the identification, packaging and delivery of the elements of a product (e.g., an executable program, documentation, release notes, or configuration data). Given that product changes can occur continually, one concern for release management is determining when to issue a release. The severity of the problems addressed by the release and measurements of the fault densities of prior releases affect this decision. The packaging task identifies which product items are to be delivered

and then selects the correct variants of those items, given the product's intended application. The information documenting the physical contents of a release is known as a version description document (VDD). The release notes describe new capabilities, known problems and platform requirements necessary for proper product operation. The package to be released also contains installation or upgrade instructions. The latter can be complicated because some users might have versions that are several releases old. In some cases, release management might be necessary to track the product's distribution to various customers or target systems (e.g., when the supplier was required to notify a customer of newly reported problems). Finally, a mechanism to help ensure the released item's integrity can be implemented (e.g., by including a digital signature).

A tool capability is needed for supporting these release management functions. For example, a connection with the tool capability supporting the CR process is useful to map release contents to the SCRs that have been received. This tool capability might also maintain information on various target platforms and customer environments.

In continuous delivery, a pipeline is established to build software continuously, as described in the previous section. The resulting artifacts from the build process include executable code and libraries, which can then be combined into an installation package and deployed into an environment for verification or production use.

7. Software Configuration Management Tools

[3*, c26s1]

Many tools can assist with CM at many levels. The scope of these tools varies depending on who uses the tools. CM is most effective when integrated with other processes and by extension with other existing tools. The selection of CM tool can be made depending on the scope that the tool is going to have.

Overview of tools:

- The configuration management system (CMS) provides enabling technology and logic to facilitate CM activities.
- Version control stores the source code, configuration files and related artifacts.
- Build automation (pipeline) is established to enable continuous delivery.
- A repository stores binaries that are created during the build process to extract the latest build artifacts and redeploy them as required — used in the release verification process.
- Configuration management database (CMDB) or similar persistence store.
- Change control tools.
- Release/deployment tools.

The CMS supports the unique identification of artifacts. Both individual artifacts and collections are specified in CM systems and related repositories. Structuring creates a logical relationship between artifacts. Validation and release establish the artifacts' integrity, as part of the release management process. Baselines are identified where stability is intended. For example, interface management is identified and controlled, making it part of the baseline process. Change management, including variants and nonconformances, is reviewed and approved, and its implementation is planned. Verification and audit activities are performed as part of the identification, change and release management process. Status and performance accounting are recorded as events occur and are made available through the CMS.

Individual support tools are typically sufficient for small organizations or development groups that do not issue variants of their software products or face other complex SCM requirements. The following are examples of these tools:

- Version control tools: These tools track, document and store individual CIs such as source code and external documentation.
- Build handling tools: In their simplest form, such tools compile and link an executable version of the software. More

advanced building tools extract the latest version from the version control software, perform quality checks, run regression tests, and produce various forms of reports, among other tasks.
- Change control tools: These tools primarily support the control of CRs and event notifications (e.g., CR status changes, milestones reached).

Project-related support tools mainly support workspace management for development teams and integrators. In addition, they can support distributed development environments. Such tools are appropriate for medium-to-large organizations that use variants of their software products and parallel development and do not have certification requirements.

Companywide-process support tools can automate portions of a companywide process, providing support for workflow management, roles and responsibilities. They can handle many items, large volumes of data, and numerous life cycles. In addition, such tools add to project-related support by supporting a more formal development process, including certification requirements.

MATRIX OF TOPICS VS. REFERENCE MATERIAL

Topic	Hass 2003 [3*]	Sommerville 2016 [4*]
1. Management of the SCM Process		
1.1. Organizational Context for SCM	Introduction	c25
1.2. Constraints and Guidance for the SCM Process	c2,c5	
1.3. Planning for SCM	c23	c25
1.3.1. SCM Organization and Responsibilities	c10-11	c25
1.3.2. SCM Resources and Schedules	c23	
1.3.3. Tool Selection and Implementation	c26s2, s6	
1.3.4. Vendor/Subcontractor Control	c13s9-c14s2	
1.3.5. Interface Control	c23s4	
1.4. SCM Plan	c23	
1.*5. Surveillance of Software Configuration Management*	c11s3	
1.5.1. SCM Measures and Measurement	c9s2; c25s2-s3	
1.5.2. In-Process Audits of SCM	c1s1	
2. Software Configuration Identification		
2.1. Identifying Items to Be Controlled	c1s2	
2.1.1. Software Configuration		

2.1.2. Software Configuration Item	c9	
2.2. Configuration Item Identifiers and Attributes	c9	
2.3. Baseline Identification		
2.4. Baseline Attributes		
2.5. Relationships Scheme Definition	c7s4	
2.6. Software Libraries	c1s3	
3. Software Configuration Change Control	c8	c25s3
3.1. Requesting, Evaluating and Approving Software Changes	c11s1	c25s3
3.1.1. Software Configuration Control Board	c11s1	c25s3
3.1.2. Software Change Request Process	c1s4, c8s4	c25s3
3.1.3. Software Change Request Forms Definition	c8s4	c25s3
3.2. Implementing Software Changes		c25s3
3.3. Deviations and Waivers		
4. Software Configuration Status Accounting	c9	
4.1. Software Configuration Status Information		
4.2. Software Configuration Status Reporting	c1s5; c9s1	
5. Software Configuration Auditing		
5.1. Software Functional Configuration Audit		
5.2. Software Physical Configuration Audit		
5.3. In-Process Audits of a Software Baseline		
6. Software Release Management and Delivery	c8s2	c25s4
6.1. Software Building		c25s2
6.2. Software Release Management		c25s2
7. Software Configuration Management Tools	c26s1	

FURTHER READINGS

S.P. Berczuk and B. Appleton, *Software Configuration Management Patterns: Effective Teamwork*, Practical Integration [6].

This book expresses useful SCM practices and strategies as patterns. The patterns can be implemented using various tools, but they are expressed in a tool-agnostic fashion.

CMMI for Development, Version 2.0 - 2.1, pp. 66–80 [7].

This model presents a collection of best practices to help software development organizations improve their processes. At maturity level 2, it suggests CM activities.

B. Aiello and L. A. Sachs, Configuration management best practices: Practical methods that work in the real world (1st edition), 2011 [8].

This book presents the seven types of change control (Chapter 4, Section 3).

REFERENCES

[1] ISO/IEC/IEEE, "ISO/IEC/IEEE 24765:2017 Systems and Software Engineering — Vocabulary," 2nd ed. 2017.

[2] IEEE. IEEE Standard 828-2012, Standard for Configuration Management in Systems and Software Engineering, 2012.

[3*] A.M.J. Hass. *Configuration Management Principles and Practices*, 1st ed. Boston: Addison-Wesley, 2003.

[4*] I. Sommerville, *Software Engineering*, 10th ed. Global ed. Pearson, 2016.

[5] J.W. Moore, *The Road Map to Software Engineering: A Standards-Based Guide*, 1st ed. Hoboken, NJ: Wiley-IEEE Computer Society Press, 2006.

[6] S.P. Berczuk and B. Appleton, *Software Configuration Management Patterns: Effective Teamwork, Practical Integration*: Addison-Wesley Professional, 2003.

[7] *CMMI for development,* Version 2.0, CMMI Institute, 2018.

[8] B. Aiello and L.A. Sachs, Configuration management best practices: Practical methods that work in the real world (1st edition), 2011.